

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

C05: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration : 1 Hour
Total Marks:50

Annexure A

Design Task

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Design Task

1. Hybrid AI Recommendation Engine Design

Design a hybrid AI-based recommendation system that integrates content-based and collaborative filtering techniques using PySpark RDDs. Explain how distributed data processing can be used to handle large-scale user-item interactions, and describe the role of intelligent agents in adapting recommendations based on user behavior and feedback in real time.

2. Smart Disaster Detection System

Develop a distributed AI system using PySpark and RDD operations to detect natural disasters (such as floods or earthquakes) from multi-source data streams (satellite images, sensors, and social media). Describe the system architecture, data fusion techniques, and how intelligent agents coordinate to provide early warnings and response strategies.

3. AI-Based Fraud Detection Framework

Design a scalable fraud detection system using PySpark RDDs to process large volumes of financial transactions. Explain how anomaly detection algorithms can be implemented in a distributed manner and how intelligent agents collaborate to identify suspicious patterns, trigger alerts, and continuously improve detection accuracy.

4. Autonomous Supply Chain Optimization System

Construct a distributed AI system using PySpark for optimizing supply chain operations (inventory, logistics, demand forecasting). Describe how RDD transformations support large-scale data processing and how multiple intelligent agents interact to make decentralized decisions for cost minimization and efficiency improvement.

5. Personalized Learning Analytics Platform

Design an AI-driven educational platform that uses PySpark and RDDs to analyze student learning behavior and performance data. Explain how the system supports adaptive learning through intelligent agents, real-time feedback, and scalable analytics to personalize content delivery for thousands of learners.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined

Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation

Distributed AI System Design Tasks

1. Hybrid AI Recommendation Engine Design

Objective

To design a scalable recommendation system that combines Content-Based Filtering and Collaborative Filtering using PySpark RDDs to provide personalized recommendations to users.

System Architecture

User-Item Interaction Data → Data Collection → PySpark RDD Layer → Content-Based Model + Collaborative Filtering Model → Hybrid AI Engine → Intelligent Agent → Personalized Recommendations → User Feedback → Model Update

PySpark RDD Processing

User-item data can be represented as (user_id, item_id, rating). PySpark RDDs distribute these records across multiple machines.

Typical RDD operations:

- `map()` – extracts user, item, and rating information.
- `filter()` – removes invalid or unwanted records.
- `groupByKey()` – groups interactions by user.
- `reduceByKey()` – aggregates ratings or interaction counts.
- `join()` – combines user-item interactions with item features.

Example:

```
ratings = sc.textFile("ratings.csv")
data = ratings.map(lambda x: x.split(", "))
user_items = data.map(lambda x: (x[0], (x[1], float(x[2]))))
popular_items = user_items.map(lambda x: (x[1][0], x[1][1])).reduceByKey(lambda a, b: a + b)
```

Hybrid Recommendation

Content-Based Filtering recommends items similar to items the user has previously liked based on item characteristics. Collaborative Filtering finds users with similar behavior and recommends items preferred by those users.

Hybrid Score = $\alpha \times \text{Content Score} + \beta \times \text{Collaborative Score}$

Distributed Processing

PySpark divides large user-item datasets into partitions. Different machines can process different partitions simultaneously, allowing the system to handle millions or billions of interactions efficiently.

Role of Intelligent Agents

Intelligent agents continuously monitor searches, clicks, ratings, purchases, watch time, and likes/dislikes. The agent updates the user's preference profile and modifies future recommendations.

Expected Result

The system provides personalized, scalable, and continuously adapting recommendations while PySpark provides distributed processing for large-scale data.

2. Smart Disaster Detection System

Objective

To develop a distributed AI system that detects disasters such as floods and earthquakes by combining satellite data, sensor information, and social-media reports.

System Architecture

Satellite Images + IoT Sensors + Social Media → Data Collection → Data Preprocessing → PySpark RDDs → Data Fusion Layer → Distributed AI Models → Disaster Detection Engine → Intelligent Agents → Early Warning and Response Strategy

Multi-Source Data

Satellite data can provide water levels, land changes, cloud formations, and damaged areas. Sensors can provide water level, temperature, ground vibration, and atmospheric pressure. Social media can provide real-time reports of flooding or earthquakes.

PySpark RDD Processing

Each data source can be converted into RDDs. RDD transformations such as `map()`, `filter()`, `flatMap()`, `reduceByKey()`, and `join()` can combine information from different sources.

Example:

```
sensor_data = sc.textFile("sensor_data.csv")
social_data = sc.textFile("social_media.csv")
sensors = sensor_data.map(lambda x: x.split(","))
social = social_data.map(lambda x: x.split(","))
```

Data Fusion

Data fusion combines evidence from multiple sources. For example, extremely high water level from sensors, a flooded region from satellite images, and multiple flood reports from social media together increase confidence that a flood has occurred.

Intelligent Agents

- Detection Agent – analyzes incoming data and identifies possible disasters.
- Verification Agent – checks whether multiple sources support the event.

- Alert Agent – sends warnings when a disaster is confirmed.
- Response Agent – suggests evacuation routes, rescue areas, and resource allocation.

Expected Result

The system provides early disaster detection, faster warning generation, and coordinated emergency response using distributed AI and intelligent agents.

3. AI-Based Fraud Detection Framework

Objective

To design a scalable fraud detection framework capable of processing millions of financial transactions using PySpark RDDs and distributed anomaly detection.

System Architecture

Financial Transactions → Data Collection → Data Preprocessing → PySpark RDD → Feature Extraction → Distributed Anomaly Detection → Fraud Detection Engine → Intelligent Agents → Alerts and Model Updates

Transaction Data

A transaction may contain Transaction ID, User ID, Amount, Location, Timestamp, Payment Method, and Merchant.

Example:

T101, U25, ₹500, Chennai

T102, U25, ₹700, Chennai

T103, U25, ₹95,000, Foreign Country

The third transaction may be suspicious because its amount and location differ significantly from normal behavior.

PySpark RDD Processing

```
transactions = sc.textFile("transactions.csv")
data = transactions.map(lambda x: x.split(","))
high_value = data.filter(lambda x: float(x[2]) > 50000)
```

Distributed Anomaly Detection

The system analyzes average transaction amount, transaction frequency, geographic changes, unusual transaction times, sudden spending increases, and multiple transactions within a short period.

Anomaly Score = Amount Score + Location Score + Frequency Score + Time Score

Transactions exceeding a defined threshold can be marked as suspicious.

Intelligent Agents

- Monitoring Agent – continuously monitors transactions.
- Anomaly Agent – identifies unusual behavior.
- Investigation Agent – combines multiple suspicious signals.
- Alert Agent – generates alerts for high-risk transactions.
- Learning Agent – uses confirmed fraud/non-fraud outcomes to improve future detection.

Expected Result

The framework can process large transaction datasets efficiently while providing real-time fraud detection, automated alerts, and continuous improvement.

4. Autonomous Supply Chain Optimization System

Objective

To construct a distributed AI system that optimizes inventory, transportation, demand forecasting, and logistics using PySpark and multiple intelligent agents.

System Architecture

Sales Data + Inventory Data + Transport Data + Supplier Data + Market Data → PySpark RDD → Distributed Analytics → Demand Forecasting and Inventory Analysis → Intelligent Agents → Optimization Engine → Final Decisions

Data Processing Using RDDs

RDDs can process historical sales, product inventory, supplier information, delivery times, transportation costs, and customer demand.

Example:

```
sales = sc.textFile("sales.csv")
sales_data = sales.map(lambda x: x.split(", "))
product_sales = sales_data.map(lambda x: (x[1], int(x[2])))
total_sales = product_sales.reduceByKey(lambda a, b: a + b)
```

RDD Transformations

- map() – transforms raw records.
- filter() – removes invalid data.
- reduceByKey() – calculates total sales for each product.
- groupByKey() – groups sales according to products or regions.
- join() – combines sales data with inventory or supplier information.

Intelligent Agents

- Demand Agent – predicts future product demand.
- Inventory Agent – determines when stock should be replenished.
- Logistics Agent – selects efficient transportation routes.

- Supplier Agent – evaluates supplier availability and delivery time.
- Cost Agent – attempts to minimize overall operational costs.

Example Workflow

Demand increases → Demand Agent detects increase → Inventory Agent checks stock → Supplier Agent checks availability → Logistics Agent selects delivery route → Optimization Agent selects the lowest-cost solution.

Expected Result

The system improves inventory management, delivery efficiency, demand prediction, supplier coordination, transportation planning, and overall cost efficiency.

5. Personalized Learning Analytics Platform

Objective

To design an AI-based educational platform that analyzes student behavior and performance using PySpark RDDs and intelligent agents to provide personalized learning.

System Architecture

Student Data → Quiz Scores + Assignment Results + Attendance + Video Watch Time + Learning Activity + Course Progress → PySpark RDD → Data Processing → Feature Extraction → Learning Analytics → Intelligent Agents → Personalized Content → Student Feedback → Continuous Adaptation

Student Data

The platform can collect quiz scores, assignment marks, course completion, time spent on topics, number of attempts, incorrectly answered questions, and learning frequency.

Example:

```
students = sc.textFile("student_data.csv")
data = students.map(lambda x: x.split(","))
scores = data.map(lambda x: (x[0], float(x[2])))
average_scores = scores.reduceByKey(lambda a, b: (a + b) / 2)
```

Learning Analytics

The system identifies strong subjects, weak subjects, frequently missed concepts, learning patterns, and students requiring additional support.

Example

Student performs poorly in Algebra → Agent identifies weak topic → Additional Algebra lessons → Practice questions → Quiz → Performance analyzed → Learning path updated.

Intelligent Agents

- Student Agent – maintains an individual learner profile.
- Assessment Agent – analyzes quiz and assignment performance.

- Recommendation Agent – selects suitable learning materials.
- Feedback Agent – provides immediate feedback.
- Progress Agent – tracks student improvement.

Adaptive Learning

High performance → Advanced content.

Average performance → Regular practice.

Low performance → Basic explanation, examples, and additional exercises.

Expected Result

The platform provides personalized learning paths, real-time feedback, scalable student analytics, and adaptive content delivery for thousands of learners.

Overall Comparison

System	Main Data	AI Function	Intelligent Agents	Main Benefit
Recommendation Engine	User-item interactions	Recommendation	User/Recommendation agents	Personalization
Disaster Detection	Sensors, satellite, social media	Event detection	Detection/Alert/Response agents	Early warning
Fraud Detection	Financial transactions	Anomaly detection	Monitoring/Alert/Learning agents	Fraud prevention
Supply Chain	Sales, inventory, logistics	Forecasting & optimization	Demand/Inventory/Logistics agents	Cost reduction
Learning Analytics	Student activity & scores	Performance prediction	Student/Feedback agents	Adaptive learning

Common PySpark RDD Workflow

LARGE-SCALE DATA → Data Collection → Data Cleaning → PySpark RDDs → RDD

Transformations (map, filter, join, reduceByKey, groupByKey) → RDD Actions (collect, count, saveAsTextFile) → AI Model → Intelligent Agents → Decision / Prediction → Feedback / Learning

Key Idea: PySpark RDDs provide the distributed data-processing layer, AI models provide prediction or optimization, and intelligent agents provide the autonomous decision-making and adaptation layer.